Prediction of Overall Survival in Patients with Myeloid Leukemia
QRT

Motawassim LAHMADI

May 2025

Dauphine | PSL
UNIVERSITÉ PARIS

Summary

1 Background **4**
1.1 Workout Machine Learning Project 4
1.2 New project : Prediction of Overall Survival in Pa tients with Myeloid Leukemia 6

2 Data description and purpose **7**
2.1 Data description 7

3 Exploratory Data Analysis (EDA) **11**
3.1 Exploring Single Variables: Univariate Insights 11
3.1.1 Molecular Dataset Analysis 11
3.1.2 Clinical Dataset Analysis 16
3.2 Variable Relationships: Bivariate Analysis 19
3.2.1 Molecular DataSet : Bivariate Analysis 19
3.2.2 Clinical DataSet : Bivariate Analysis 23

4 Data Preprocessing and Quality Assurance **25**
4.1 Molecular dataset Preprocessing 25
4.2 Clinical dataset Preprocessing 29

5 Feature Selection: Identifying Key Prognostic Factors **34**
5.1 Variance Threshold 34
5.2 Pearson Correlation 34

6 Survival Analysis **34**
6.1 Key Concepts 35
6.2 Common Models 35
6.3 Examples 35
6.4 Overall Survival of Myeloid Leukemia Patients 36

7 Cox Proportional Hazards Model **36**
7.1 Mathematical understanding 36
7.2 CoxPHFitter 36
7.3 Computation of p-values in Cox Proportional Hazards Model . . 37
7.4 Concordance Index in Survival Analysis 38
7.5 Integrated Brier Score (IBS) 39

8 Evaluation of different survival models **41**
8.1 Models 41
8.2 Concordance Index (IPCW) Evaluation Results 42
8.3 HyperParameters tuning with GridSearchCV 42

9 Clinical Interpretation and Survival Curve Analysis **45**
9.1 Survival Curve 45
9.2 Male Female Survival Curve 46
9.3 Down Syndrome Survival 47
9.4 Deletion anomaly Survival 48

1 Background

1.1 Workout Machine Learning Project

Before starting this project, I had already initiated a study related to sports. Specifically, the goal was to predict post-exercise mood based on various factors such as training duration and hours of sleep. Below is a glimpse of the dataset used in that study.
Here's what the dataset looked like :

shape: (5, 20)

User ID	Age	Gender	Height (cm)	…	VO2 Max	Body Fat (%)	Mood Before Workout	Mood After Workout
---	---	---	---		---	---	---	---
i64	i64	str	i64		f64	f64	str	str
1	39	Male	175	…	38.4	28.5	Tired	Fatigued
2	36	Other	157	…	38.4	28.5	Happy	Energized
3	25	Female	180	…	38.4	28.5	Happy	Fatigued
4	56	Male	154	…	38.4	28.5	Neutral	Neutral
5	53	Other	194	…	38.4	28.5	Stressed	Energized

Therefore, we will visualize the numerical data to better understand the underlying patterns.

Mood Before Workout

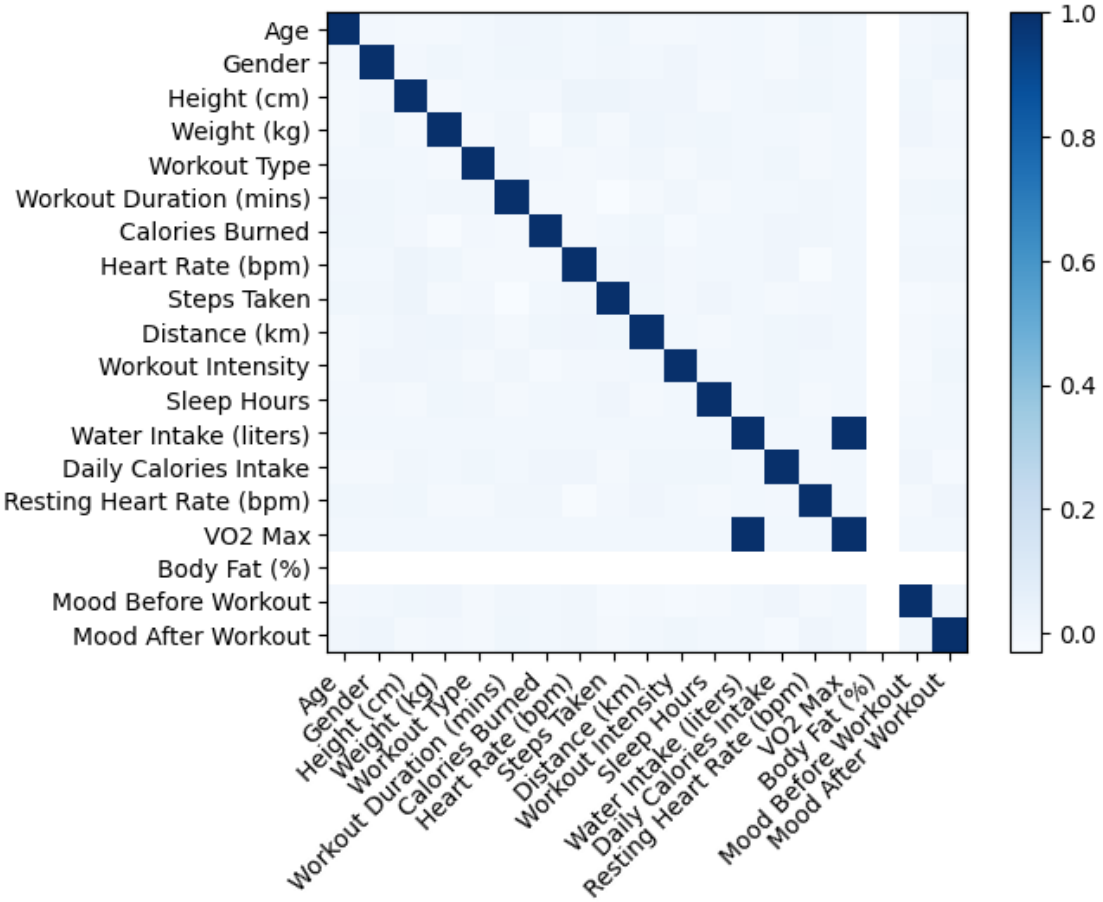
- Very low correlation with all other variables (values close to 0).
- This means that no other variable (age, sleep, intensity, calories, etc.) adequately explains this pre-workout mood.

Mood After Workout

- Same thing: very low linear correlation with other variables, including those we might intuitively believe to be related (Workout Intensity, Calories Burned, etc.).
- No clear trend emerges to justify a reliable prediction.

A univariate analysis shows that the data are poorly distributed, or at least randomly distributed. We find, for example, that the heart rate during exercise is perfectly distributed despite the difference in physical effort. Each person has the same daily water intake, and the same applies to VO2 Max. What's more, for every sporting activity such as yoga, there are always as many Neutral, Energetic and Fatigued athletes, which makes it impossible to discriminate on the basis of this variable.

Consequence : A predictive model will have difficulty generalizing correctly because the data are strongly biased towards certain classes, and transitions between moods are not linear.



<u>Remark :</u> This dataset was **synthetically** generated to simulate real-world fitness tracking data. The data mimics information collected from various fitness apps, smartwatches, and wearable devices such as Apple Watch, Fitbit, Garmin, and Samsung Health

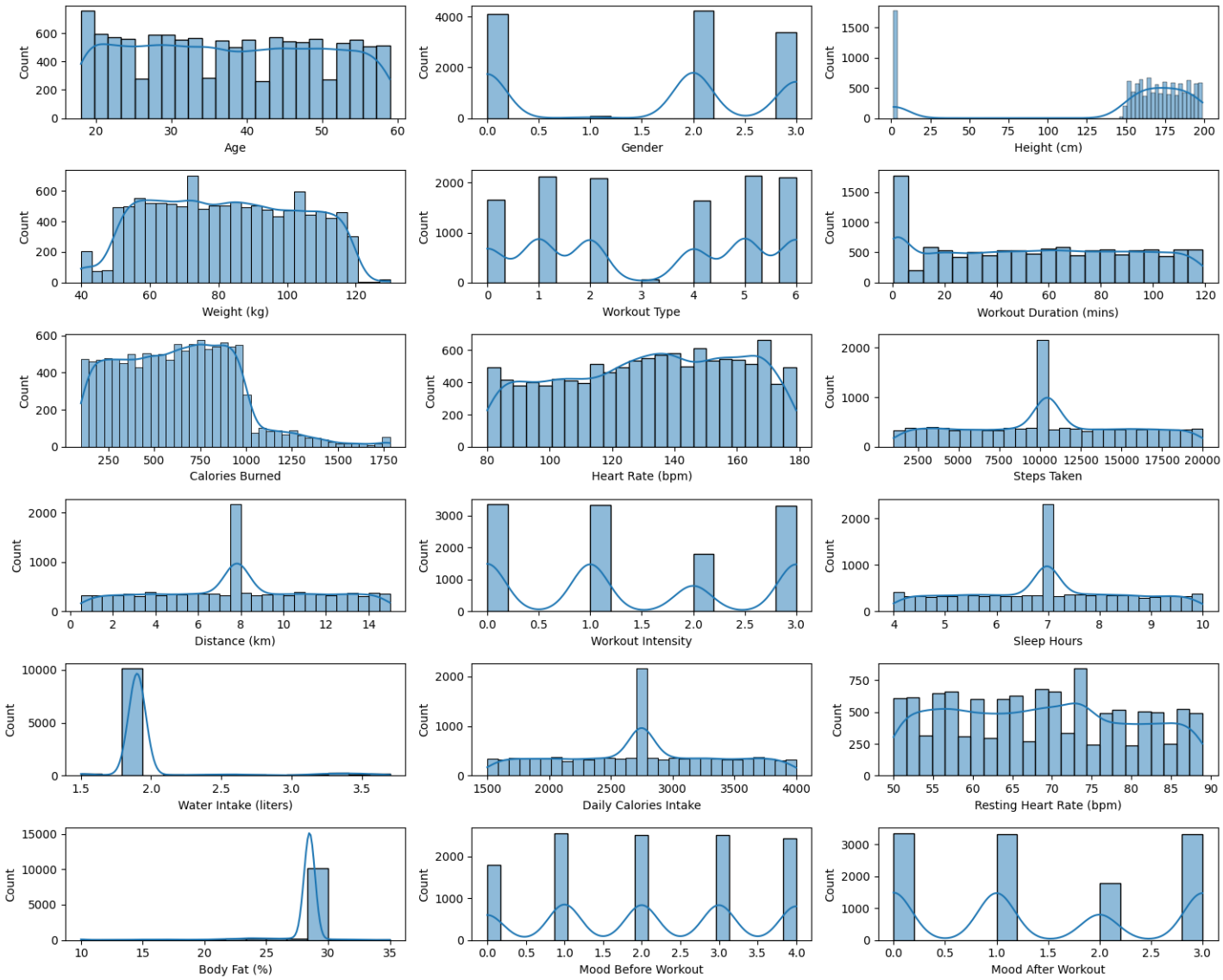


Figure 1: Univariate Analysis : Workout Dataframe

1.2 New project : Prediction of Overall Survival in Patients with Myeloid Leukemia

So I decided to start a new project. This time, I took a problem directly from the Data Challenge website.

- **Context** : In recent years, the medical sector has increasingly adopted methods based on the analysis of large quantities of health data, particularly in the field of prognosis and treatment of complex diseases such as cancer. Predictive healthcare models have transformed patient care, enabling far more tailored and effective treatment strategies. These advances are particularly valuable in oncology, where accurate predictive models can significantly improve the quality and timing of therapeutic decisions.

- **Goal** : In partnership with the Institut Gustave Roussy, this year's QRT Data Challenge focuses on predicting the risk of death for patients diagnosed with a blood cancer, specifically a subtype of adult myeloid leukemia. For these patients, risk assessment is measured by overall survival - the period from initial diagnosis to the patient's death or last recorded follow-up.

- **Why is important ?** Estimating a patient's prognosis is essential for tailoring the therapeutic approach. Patients identified as having a low-risk profile may receive supportive therapies aimed at improving blood parameters and overall quality of life, while patients identified as high-risk may be prioritized for more intensive treatment options, such as hematopoietic stem cell transplantation. Accurate risk predictions could therefore lead to better clinical decisions, improved quality of life for patients, and more efficient use of resources within healthcare facilities.

2 Data description and purpose

2.1 Data description

The dataset is divided into three :

- A **Molecular** dataset.
- A **Clinical** dataset.
- A **Survival** dataset.

- Molecular Dataset

shape: (5, 11)

ID	CHR	START	END	…	PROTEIN_CHANGE	EFFECT	VAF	DEPTH
---	---	---	---		---	---	---	---
str	str	f64	f64		str	str	f64	f64
P100000	11	1.19149248e8	1.19149248e8	…	p.C419Y	non_synonymous_codon	0.083	1308.0
P100000	5	1.31822301e8	1.31822301e8	…	p.Y164*	stop_gained	0.022	532.0
P100000	3	7.769406e7	7.769406e7	…	p.?	splice_site_variant	0.41	876.0
P100000	4	1.06164917e8	1.06164917e8	…	p.R1262L	non_synonymous_codon	0.43	826.0
P100000	2	2.5468147e7	2.5468163e7	…	p.E505fs*141	frameshift_variant	0.0898	942.0

Figure 2: Molecular dataframe

Data types

- **ID** : Unique identifier for each patient.
- **CHR, START, END**: Chromosomal position of the mutation in the human genome.
- **REF, ALT**: Reference and alternative (mutant) nucleotides.
- **GENE**: The gene affected by the mutation.
- **PROTEIN CHANGE**: Impact of the mutation on the protein produced by the gene.
- **EFFECT**: General classification of the mutation's impact on gene function.
- **VAF**: Variant Allelic Fraction, representing the proportion of cells carrying the mutation.

- Clinical Dataset

shape: (5, 9)

ID	CENTER	BM_BLAST	WBC	…	MONOCYTES	HB	PLT	CYTOGENETICS
---	---	---	---		---	---	---	---
str	str	f64	f64		f64	f64	f64	str
P132697	MSK	14.0	2.8	…	0.7	7.6	119.0	46,xy,del(20)(q12)[2]/46,xy[18…
P132698	MSK	1.0	7.4	…	0.1	11.6	42.0	46,xx
P116889	MSK	15.0	3.7	…	0.1	14.2	81.0	46,xy,t(3;3)(q25;q27)[8]/46,xy…
P132699	MSK	1.0	3.9	…	0.1	8.9	77.0	46,xy,del(3)(q26q27)[15]/46,xy…
P132700	MSK	6.0	128.0	…	0.9	11.1	195.0	46,xx,t(3;9)(p13;q22)[10]/46,x…

Figure 3: Clinical Dataframe

Data types

- **ID** : Unique identifier for each patient.
- **CENTER**: The clinical center where the patient is treated.
- **BM BLAST**: Percentage of blasts in bone marrow, indicating the proportion of abnormal blood cells found in patients' bone marrow.
- **WBC**: White blood cell count, measured in Giga/L (blood parameter).
- **ANC**: Absolute number of neutrophils in blood, in Giga/L (blood parameter).
- **MONOCYTES**: Number of monocytes in blood, in Giga/L (blood parameter).
- **HB**: Hemoglobin level, measured in g/dL (blood parameter).
- **PLT** : Platelet count, in Giga/L (blood parameter).
- **CYTOGENETICS** Detailed description of the patient's karyotype, representing chromosomal abnormalities detected in blood cancer cells. Notations follow the ISCN standard, with typical markers such as 46,XX for a normal female karyotype or 46,XY for a normal male karyotype. Abnormalities such as monosomy 7 (loss of chromosome 7) indicate high-risk disease.

- Target Dataset

ID	OS_YEARS	OS_STATUS
str	f64	f64
P132697	1.115068	1.0
P132698	4.928767	0.0
P116889	2.043836	0.0
P132699	2.476712	1.0
P132700	3.145205	0.0

Figure 4: Target Dataframe

Data types

- **ID** : Unique identifier for each patient.
- **OS YEARS**: Overall survival time in years since diagnosis.
- **OS STATUS**: Survival status indicator, where 1 indicates death and 0 indicates that the patient was alive at the last follow-up.

shape: (9, 4)

statistic	ID	OS_YEARS	OS_STATUS
str	str	f64	f64
count	3323	3173.0	3173.0
null_count	0	150.0	150.0
mean	null	2.480713	0.504255
std	null	2.588259	0.500061
min	P100000	0.0	0.0
25%	null	0.652055	0.0
50%	null	1.652055	1.0
75%	null	3.572603	1.0
max	P132729	22.043836	1.0

Figure 5: Target description

3 Exploratory Data Analysis (EDA)

3.1 Exploring Single Variables: Univariate Insights

3.1.1 Molecular Dataset Analysis

Molecular Dataframe

shape: (5, 11)

ID	CHR	START	END	…	PROTEIN_CHANGE	EFFECT	VAF	DEPTH
---	---	---	---		---	---	---	---
str	str	f64	f64		str	str	f64	f64
P100000	11	1.19149248e8	1.19149248e8	…	p.C419Y	non_synonymous_codon	0.083	1308.0
P100000	5	1.31822301e8	1.31822301e8	…	p.Y164*	stop_gained	0.022	532.0
P100000	3	7.769406e7	7.769406e7	…	p.?	splice_site_variant	0.41	876.0
P100000	4	1.06164917e8	1.06164917e8	…	p.R1262L	non_synonymous_codon	0.43	826.0
P100000	2	2.5468147e7	2.5468163e7	…	p.E505fs*141	frameshift_variant	0.0898	942.0

Molecular Dataframe description

shape: (9, 12)

statistic	ID	CHR	START	…	PROTEIN_CHANGE	EFFECT	VAF	DEPTH
---	---	---	---		---	---	---	---
str	str	str	f64		str	str	f64	f64
count	10935	10821	10821.0	…	10923	10935	10846.0	10821.0
null_count	0	114	114.0	…	12	0	89.0	114.0
mean	null	null	8.0783e7	…	null	null	0.305087	1051.229554
std	null	null	5.6427e7	…	null	null	0.211524	552.861902
min	P100000	1	394899.0	…	FLT3_ITD	2KB_upstream_variant	0.02	16.0
25%	null	null	3.1022441e7	…	null	null	0.1025	660.0
50%	null	null	7.4732959e7	…	null	null	0.3213	975.0
75%	null	null	1.15258747e8	…	null	null	0.442	1353.0
max	P132729	X	2.26252135e8	…	p.Y974fs*8	synonymous_codon	0.999	7156.0

Figure 6: Molecular description

Remark : The "Depth" variable has a very high standard variance. In addition, the "START" and "END" **columns have many similar values**. (We'll use these remarks during pre-processing).

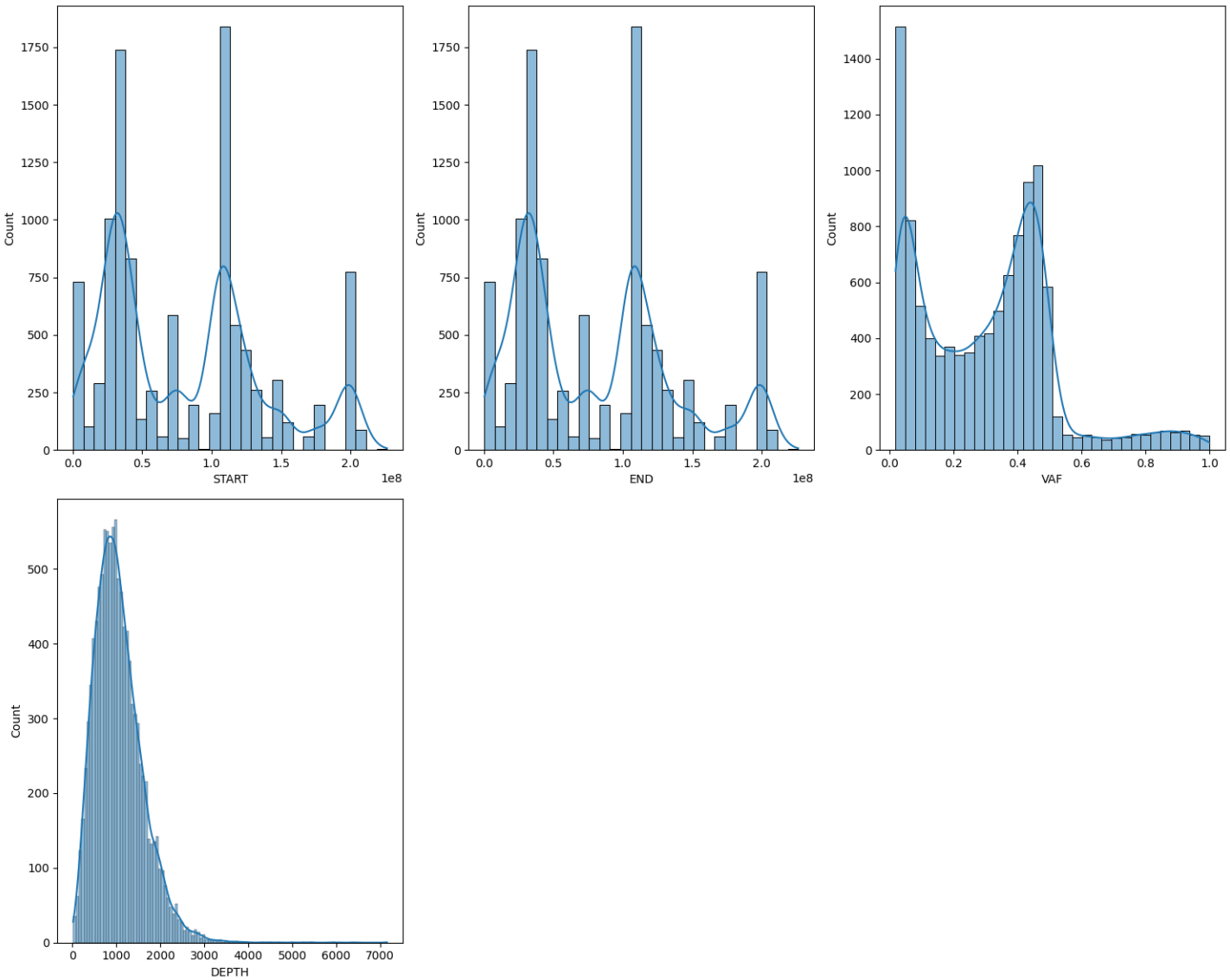


Figure 7: Numerical Analysis

<u>Remark :</u> If we analyze the graphs, **we can assume that the "VAF" and "DEPTH" variable have quite a few outliers**.
In addition, we can observe that the "DEPTH" variable seems to follow **a binomial distribution**.

Finally, we confirm our hypothesis concerning the similarity of "START" and "END" variables.

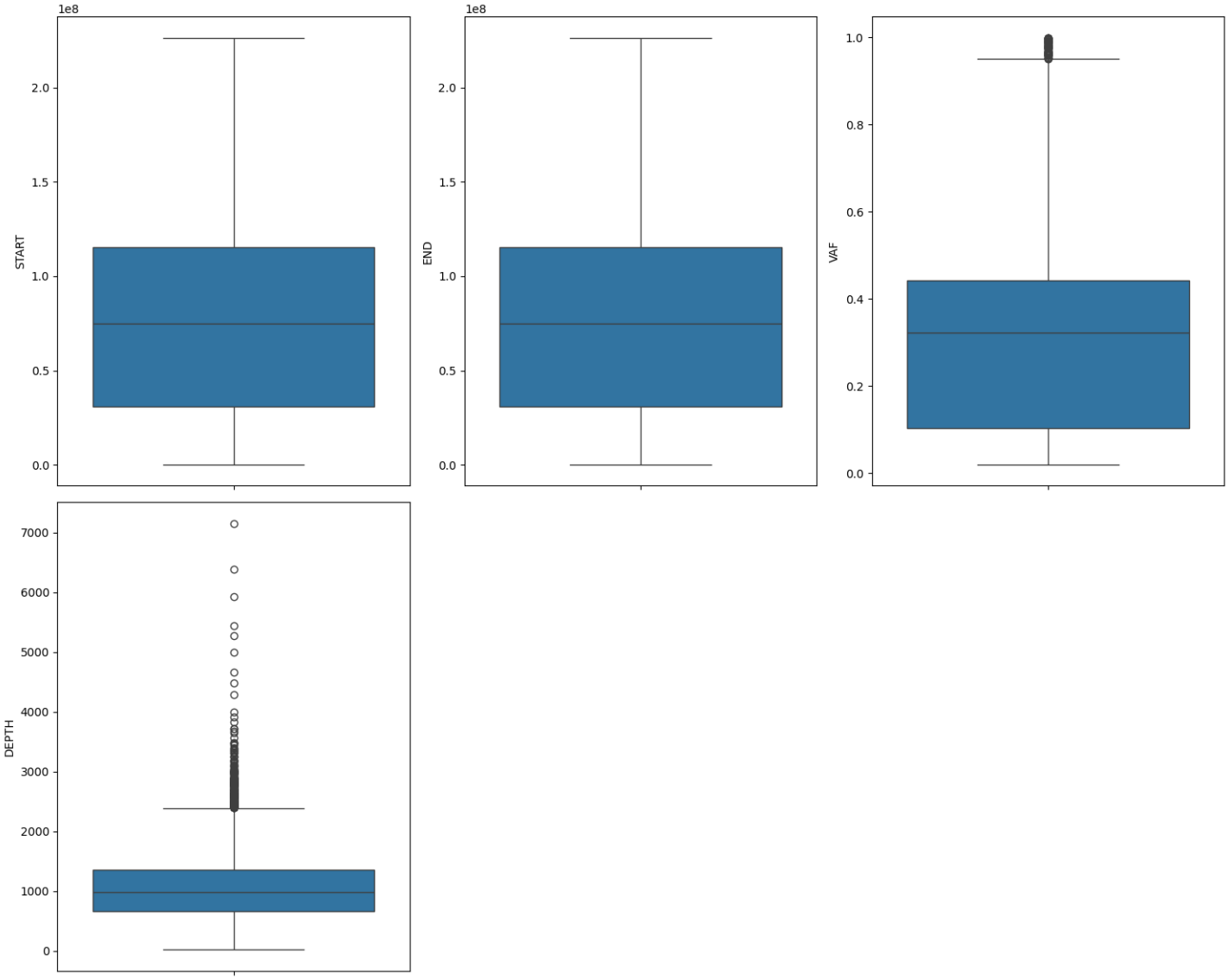


Figure 8: BoxPlot

Remark : After analyzing the boxplot, we can clearly see many points above the "Whisker".
Using the IQR Method (InterQuartile Method) we can suppose that theses points are **outliers**.

In addition, the data are highly dispersed in the VAF variable, as the median-Q1 and median-Q3 gaps are very different.

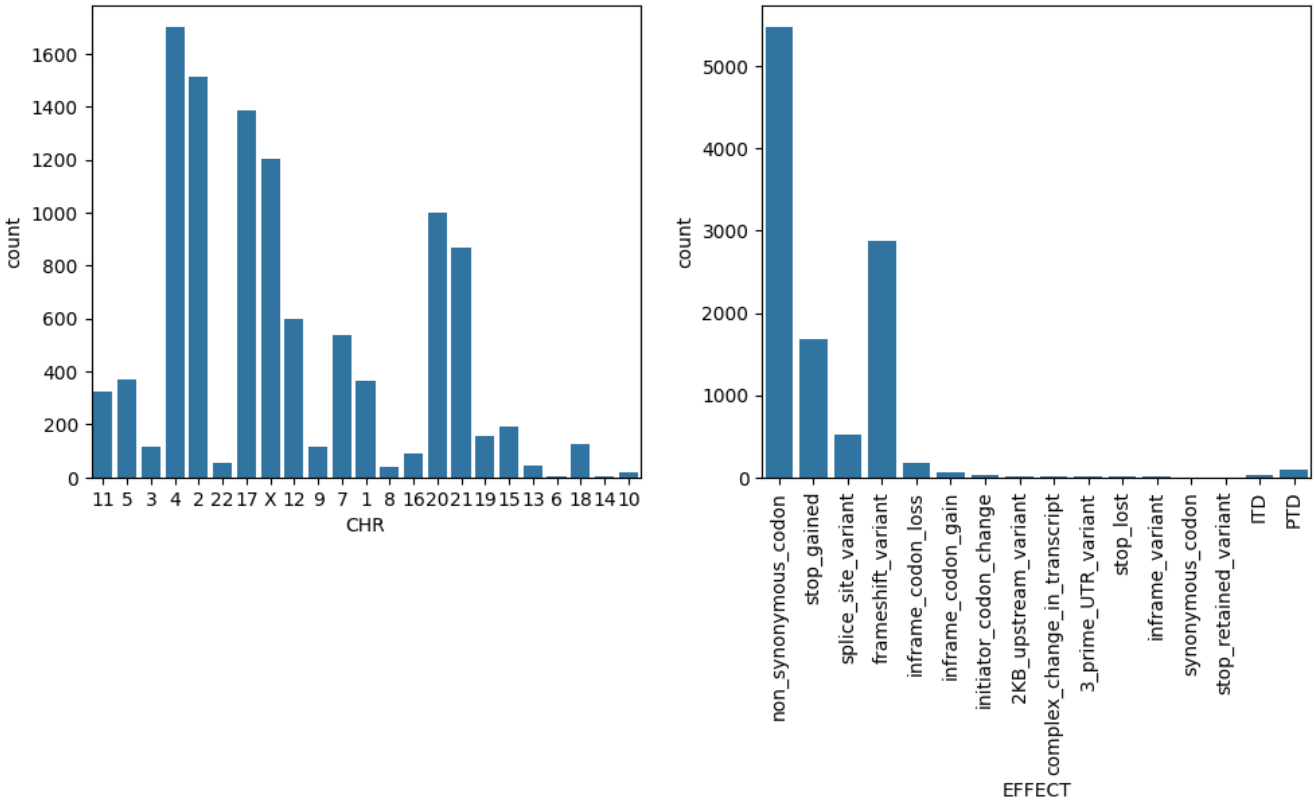


Figure 9: Categorical Analysis

<u>Remark :</u> We notice an "X" variable in CHR column that has an important count . <u>Thus , we need to find its meaning .</u>

shape: (5, 2)

GENE	count
---	---
str	u32
TET2	1663
ASXL1	951
SF3B1	775
DNMT3A	604
RUNX1	578

shape: (5, 2)

PROTEIN_CHANGE	count
---	---
str	u32
p.?	529
p.K700E	426
p.G646fs*12	370
p.P95H	246
p.P95L	148

shape: (5, 2)

EFFECT	count
---	---
str	u32
non_synonymous_codon	5471
frameshift_variant	2877
stop_gained	1673
splice_site_variant	512
inframe_codon_loss	168

shape: (5, 2)

REF	count
---	---
str	u32
C	3199
G	2965
T	1771
A	1234
TC	123

shape: (5, 2)

ALT	count
---	---
str	u32
T	3495
A	2541
C	1817
G	1336
AG	414

Figure 10: The five most common categorical elements

3.1.2 Clinical Dataset Analysis

Clinical Dataframe

shape: (5, 9)

ID	CENTER	BM_BLAST	WBC	…	MONOCYTES	HB	PLT	CYTOGENETICS
---	---	---	---		---	---	---	---
str	str	f64	f64		f64	f64	f64	str
P132697	MSK	14.0	2.8	…	0.7	7.6	119.0	46,xy,del(20)(q12)[2]/46,xy[18…
P132698	MSK	1.0	7.4	…	0.1	11.6	42.0	46,xx
P116889	MSK	15.0	3.7	…	0.1	14.2	81.0	46,xy,t(3;3)(q25;q27)[8]/46,xy…
P132699	MSK	1.0	3.9	…	0.1	8.9	77.0	46,xy,del(3)(q26q27)[15]/46,xy…
P132700	MSK	6.0	128.0	…	0.9	11.1	195.0	46,xx,t(3;9)(p13;q22)[10]/46,x…

Clinical Dataframe description

shape: (9, 10)

statistic	ID	CENTER	BM_BLAST	…	MONOCYTES	HB	PLT	CYTOGENETICS
---	---	---	---		---	---	---	---
str	str	str	f64		f64	f64	f64	str
count	3323	3323	3214.0	…	2722.0	3213.0	3199.0	2936
null_count	0	0	109.0	…	601.0	110.0	124.0	387
mean	null	null	5.982545	…	0.955868	9.893549	167.0489	null
std	null	null	7.615439	…	2.666478	2.041158	149.477031	null
min	P100000	CCH	0.0	…	0.0	4.0	2.0	+8(fish)
25%	null	null	1.0	…	0.15	8.5	66.0	null
50%	null	null	3.0	…	0.37	9.7	123.0	null
75%	null	null	8.0	…	0.784	11.2	230.0	null
max	P132729	VU	91.0	…	44.2	16.6	1451.0	tris8

Figure 11: Clinical Description

Remark : The "PLT" variable has a high standard deviation

shape: (5, 2)

CENTER	count
---	---
str	u32
KI	900
DUS	455
PV	316
GESMD	246
RMCN	199

shape: (5, 2)

CYTOGENETICS	count
---	---
str	u32
46,xy[20]	432
null	387
46,xy	276
46,xx[20]	254
46,xx	192

Figure 12: The five most common categorical elements

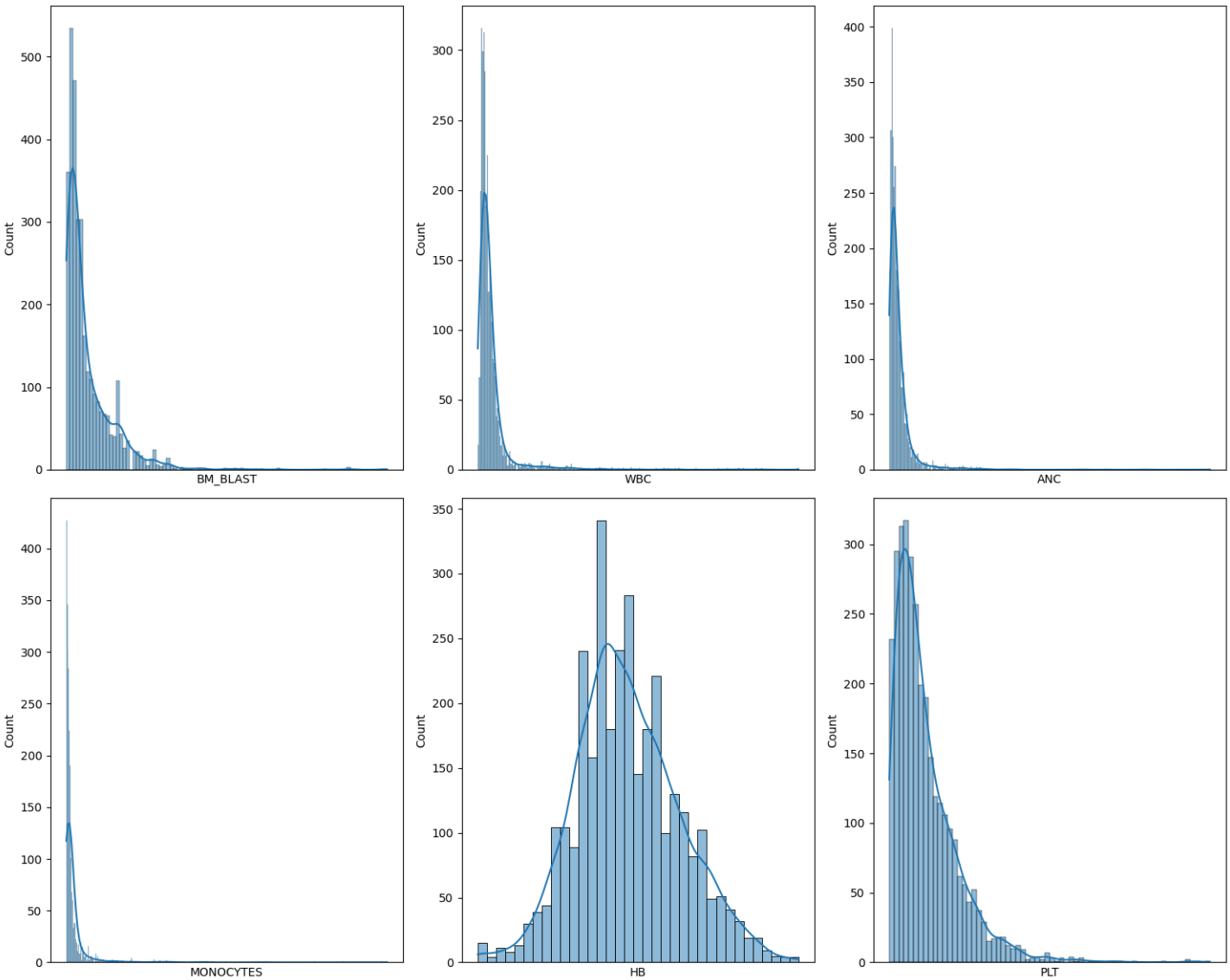


Figure 13: Numerical HistPlot

<u>Remark :</u> Every variable seems to follow a **specific distribution** . For instance, HB seems to follow **<u>a Gaussian while PLT more of a Binomial.</u>**

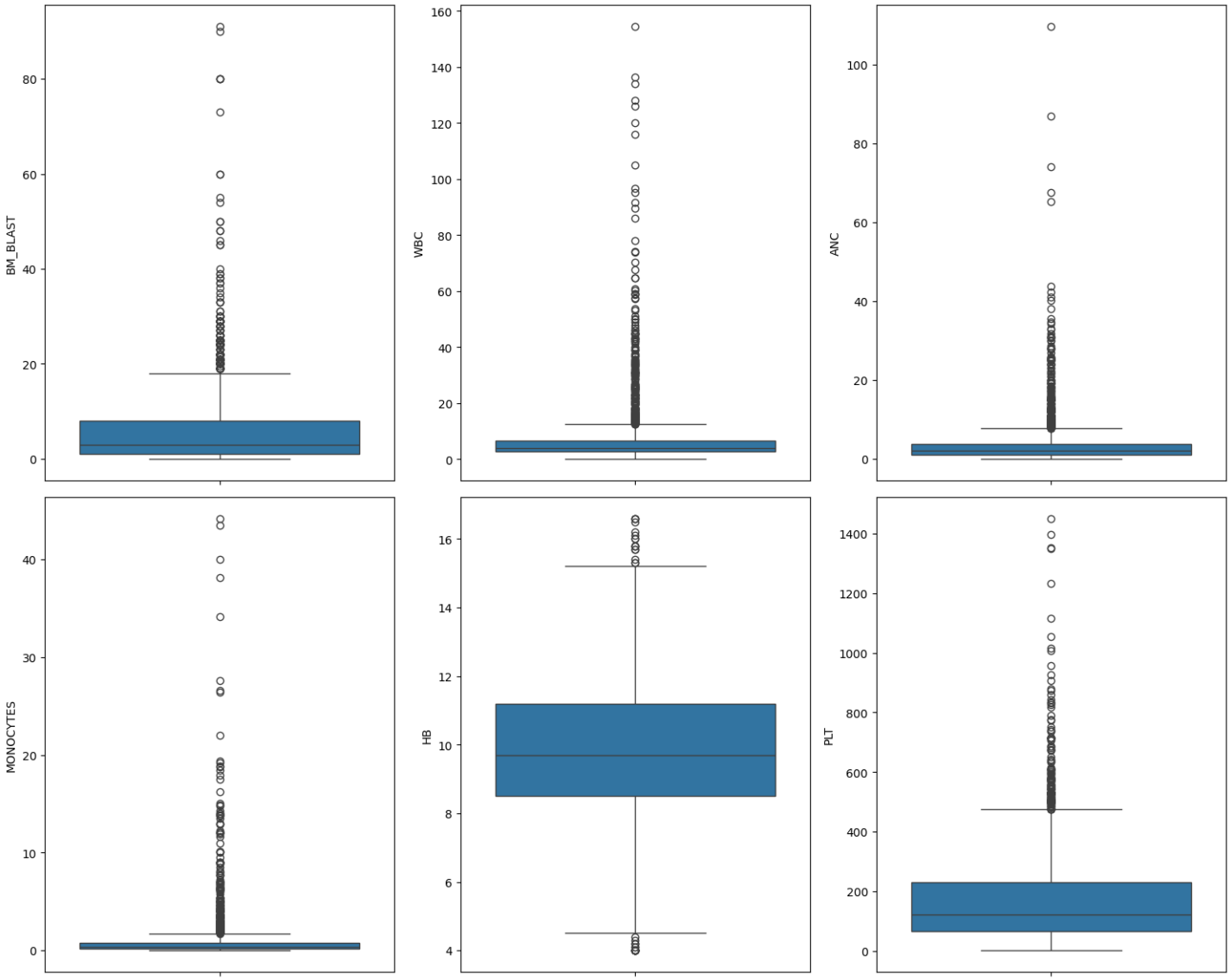


Figure 14: Numerical BoxPlot

Remark : Monocytes , WBC and ANC have small interquartile range and a lot of outliers.

3.2 Variable Relationships: Bivariate Analysis

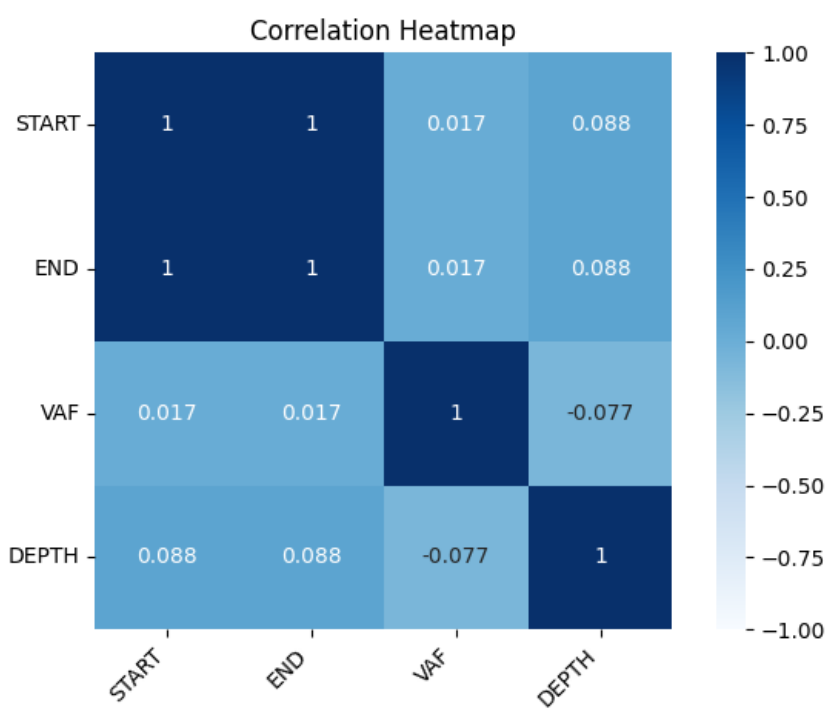
3.2.1 Molecular DataSet : Bivariate Analysis

We will begin by studying the correlation between the numerical variables . To do this, we'll plot the correlation matrix using Pearson's coefficient.

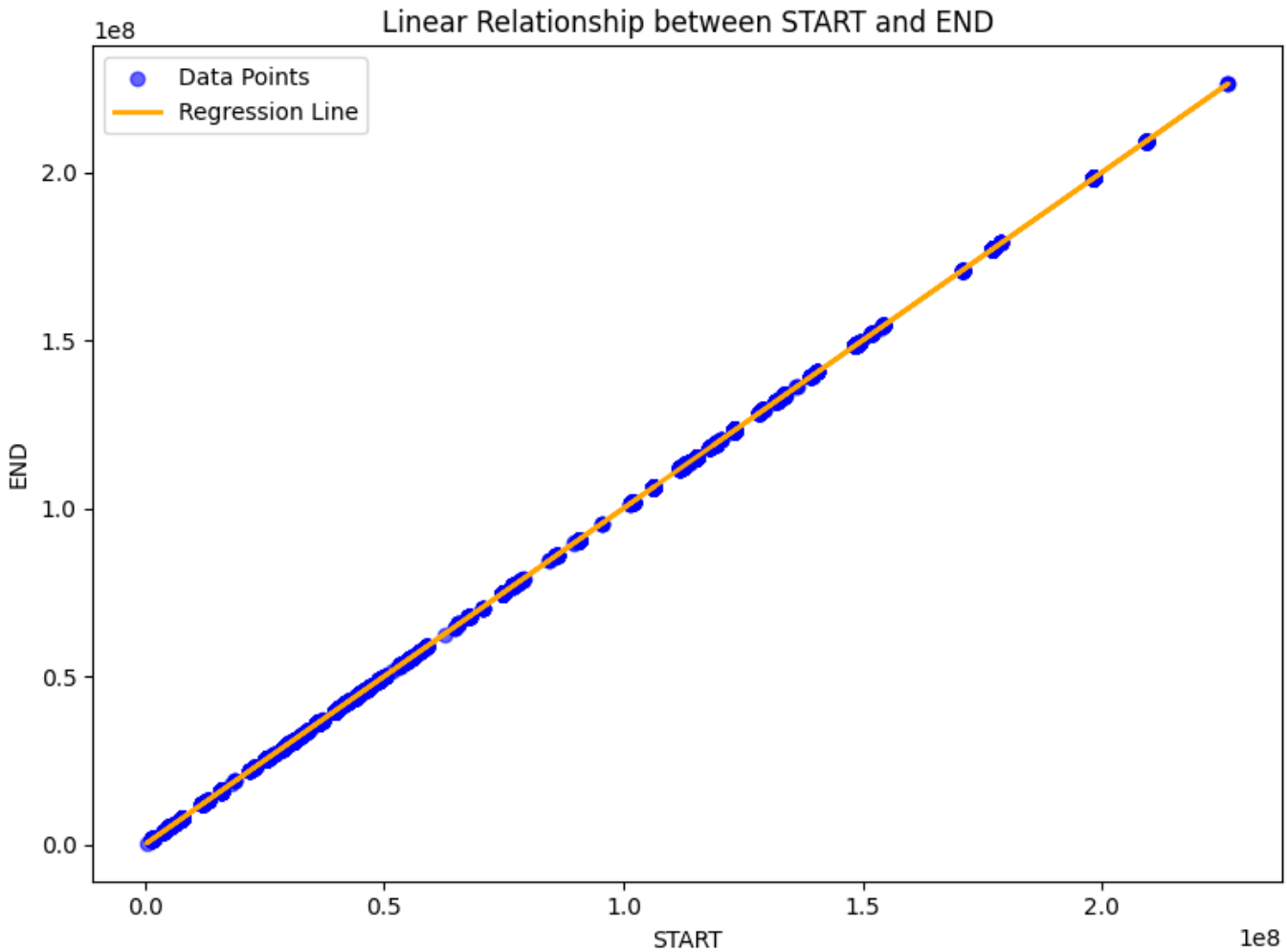
Remark : Pearson correlation is a number that tells us how strongly **two values are linearly related.**

It gives a result between -1 and +1:

- +1 : **Perfect positive** relationship (both increase together)
- -1 = **Perfect negative** relationship (one increases, the other decreases)
- 0 = **No linear relationship**

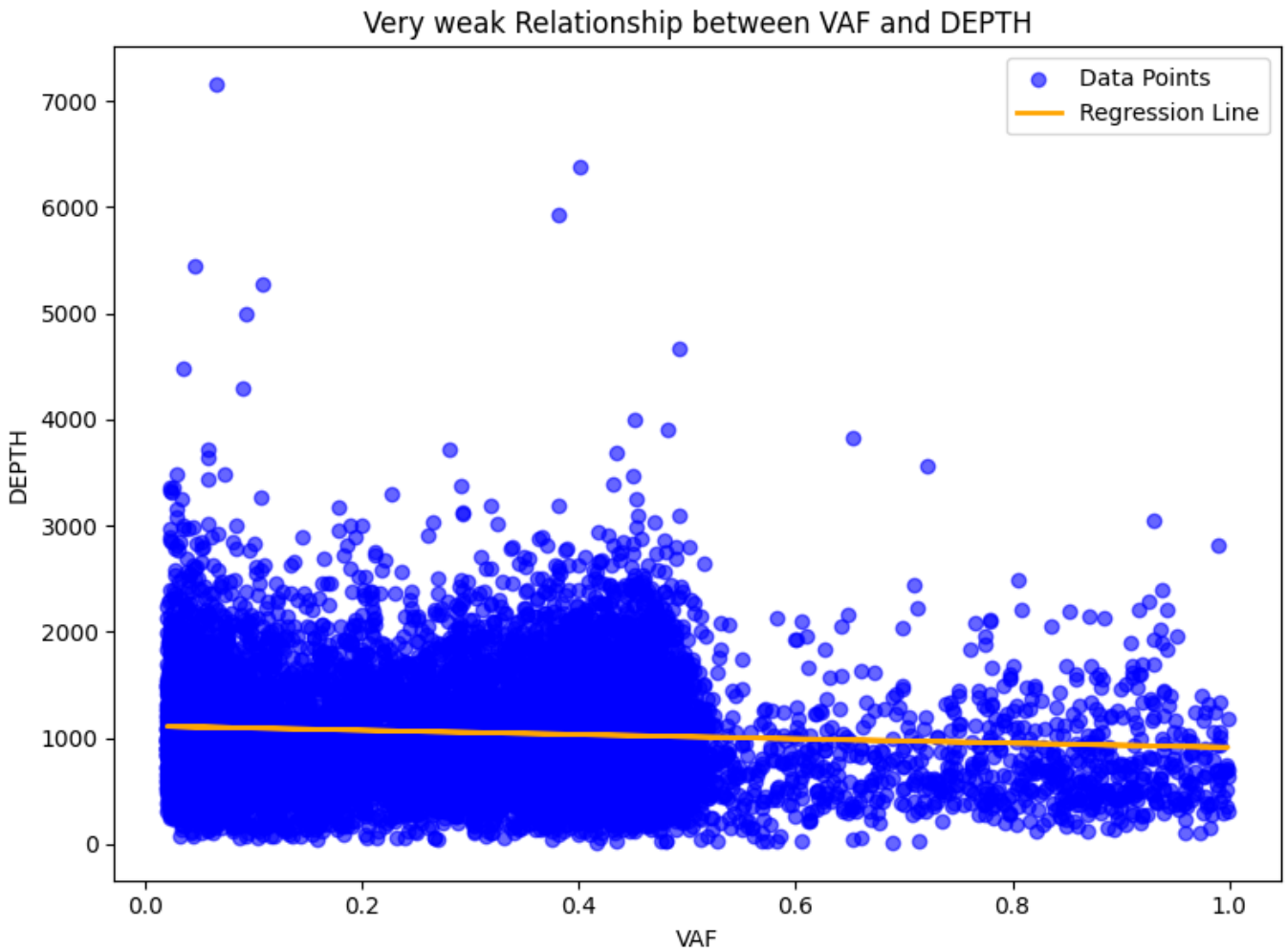


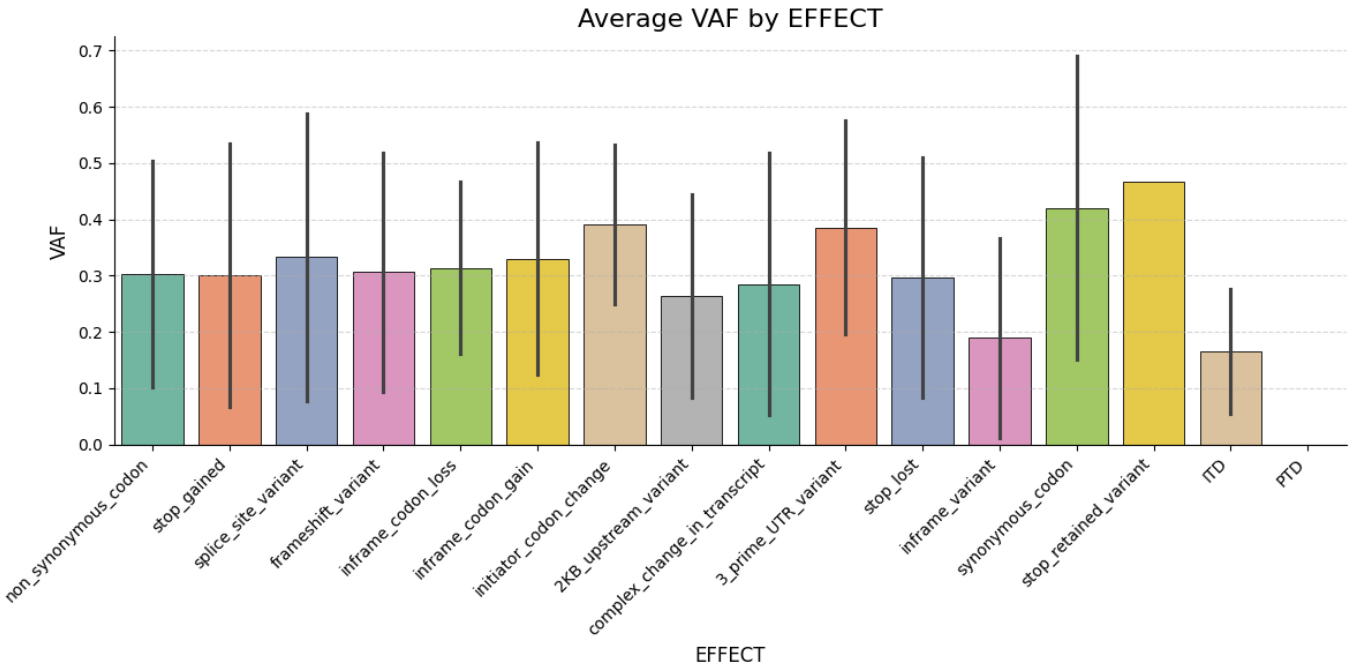
Remark : We notice that START and END variables have **a perfect positive relationship.**

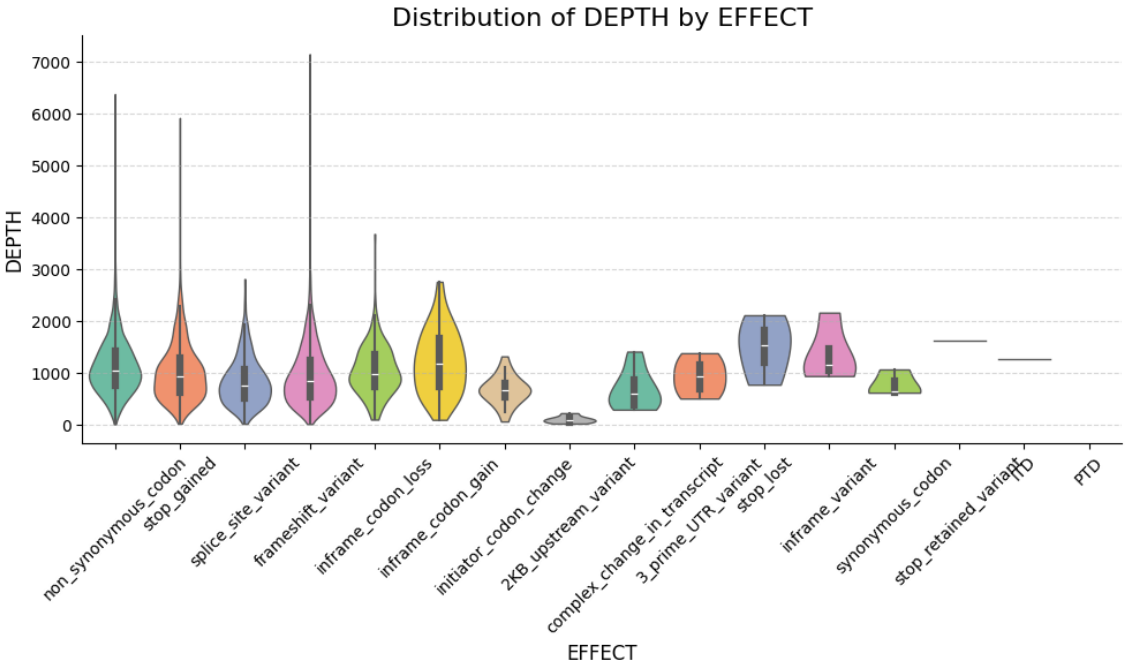


<u>Remark :</u> Now we can assume that we can later **drop one of the two variables in our feature selection.**

We will now look at a non-linear linear relationship in our molecular dataset.

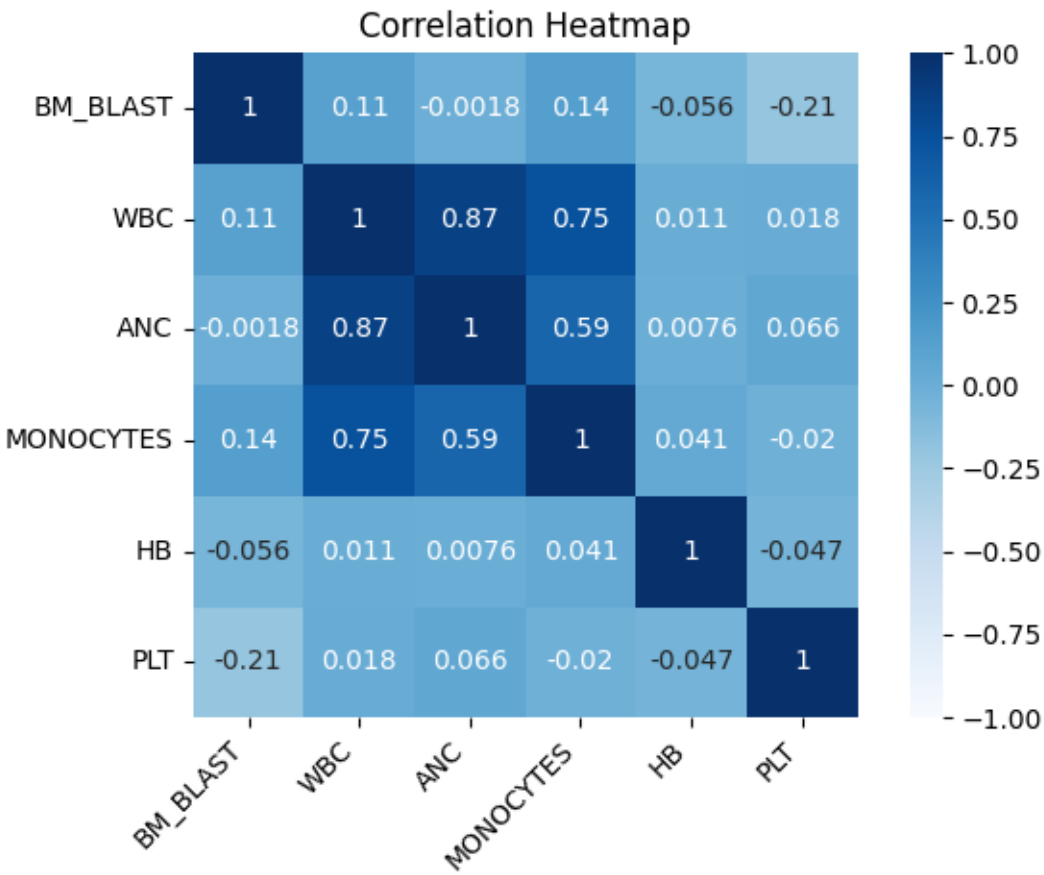



Average VAF by EFFECT
VAF
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
non_synonymous_codon
stop_gained
splice_site_variant
frameshift_variant
inframe_codon_loss
inframe_codon_gain
initiator_codon_change
2KB_upstream_variant
complex_change_in_transcript
3_prime_UTR_variant
stop_lost
inframe_variant
synonymous_codon
stop_retained_variant
ITD
PTD
EFFECT

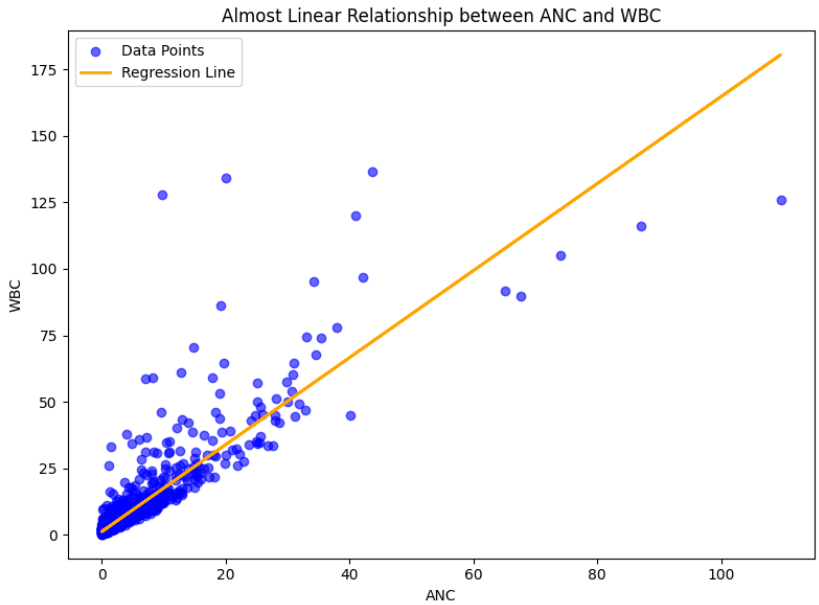

Distribution of DEPTH by EFFECT
DEPTH
0
1000
2000
3000
4000
5000
6000
7000
non_synonymous_codon
stop_gained
splice_site_variant
frameshift_variant
inframe_codon_loss
inframe_codon_gain
initiator_codon_change
2KB_upstream_variant
complex_change_in_transcript
3_prime_UTR_variant
stop_lost
inframe_variant
synonymous_codon
stop_retained_variant
ITD
PTD
EFFECT

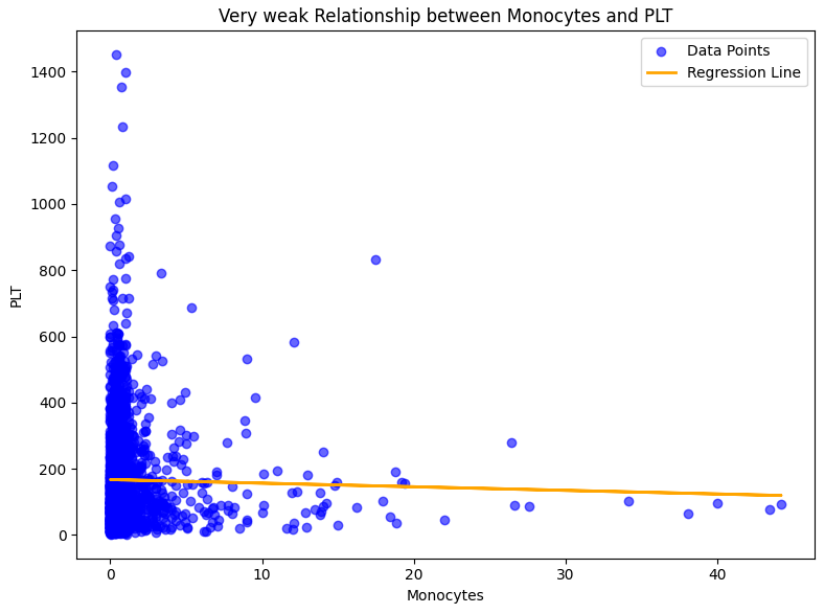
3.2.2 Clinical DataSet : Bivariate Analysis

We will begin by studying the correlation between the numerical variables . To do this, we'll plot the correlation matrix using **Pearson's coefficient.**



Remark : We can observe that ANC and WBC have a high positive correlation compared to Monocytes and PLT.


Almost Linear Relationship between ANC and WBC
Data Points
Regression Line
WBC
ANC
0
25
50
75
100
125
150
175
0
20
40
60
80
100


Very weak Relationship between Monocytes and PLT
Data Points
Regression Line
PLT
Monocytes
0
200
400
600
800
1000
1200
1400
0
10
20
30
40

4 Data Preprocessing and Quality Assurance

In this section, we'll look at the strategies we've implemented to pre-process the various dataframes. **From imputing null values to understanding the different columns.**

4.1 Molecular dataset Preprocessing

CHR : Chromosomal position of the mutation in the human genome.

Reminder : Human cells contain 23 pairs of chromosomes (i.e. 46 chromosomes). Twenty-two pairs are made up of 2 identical chromosomes, called autosomes. The twenty-third pair is made up of sex chromosomes, XX in women and XY in men. **The twenty-third chromosome is often designated directly by an X.**

In our molecular dataset , the CHR variable is a **string variable.** We created for this a function that casts the string to its integer with **a special transformation for the X value .**

```
    if chromosome_str.upper() == 'X':
        return 23  # X chromosome is represented as 23
    else:
        try:
            return int(chromosome_str)
        except ValueError:
            raise ValueError(f"Cannot convert '{
                chromosome_str}' to integer")
```

REF and ALT : Reference and alternative (mutant) nucleotides.

Definition : A nucleotide is a chemical structure composed of a nitrogenous base, a phosphate and a sugar.

Property : There are 4 types of nucleotides (also called bases) symbolized by **the letters A, C, G and T respectively named Adenine, Cytosine, Guanine and Thymine.**

The challenge with the REF and ALT columns **lies in interpreting the diverse values they contain in a biologically meaningful way.** These columns include zeros, as well as nucleotide sequences ranging in length from 1 to 500 bases. **Therefore, it is essential to categorize these variations into meaningful groups to better understand their biological significance.**

Example :

- REF : "ACAGGTGAGGCACCTGAGATTAGAGCAGGA..."

To this end, we have adapted several strategies to pre-process these two particular columns.

Different functions :

- `def is_complex_nucleotide(ref,alt)`
- `REF_Length`
- `ALT_Length`
- `def is_transition(ref,alt)`
- `def is_transversion(ref,alt)`
- `def is_indel(ref,alt)`
- `def is_snv_REF(ref)`
- `def is_snv_ALT(alt)`

Explanation :

Single nucleotide variant : A DNA sequence variation that occurs when a single nucleotide (adenine, thymine, cytosine, or guanine) in the genome sequence is altered.

Complex nucleotide : A nucleotide is considered complex if the length of the reference and alternative nucleotides is strictly greater than 1. We have also created two new columns containing **the length of the various nucleotides.**

Transition : In molecular biology, a transition is a DNA point mutation corresponding to **the replacement of either one pyrimidine (A or G) by the other pyrimidine, or one purine (T or C) by the other purine.**
Example of a transition : $A \to G$ or $C \to T$

Transversion: In molecular biology, transversion refers to a DNA point mutation in **which a purine (A or G) is replaced by a pyrimidine (T or C), or vice versa.**

Example of a tranversion : $A \to C$ or $C \to G$

Insertion or Deletion : There has been an insertion or deletion of nucleotides when the size of the reference nucleotide and the alternating nucleotide are different.

Protein Change : Impact of the mutation on the protein produced by the gene.

Example :

- p.C419Y
- MLL_PTD

The challenge here is to dissect this nomenclature.

- How to read a protein change :

- p.E505fs*141 :

- p : This indicates a protein-level change
- E : The letter E refers to a particular Amino acids . For instance here it is the Glutamic acid.
- 505 : The position of the original amino acid in the protein sequence.
- fs : Frameshift. This means there is **a frameshift mutation** starting at this position.
- The new, abnormal reading frame continues for 141 amino acids before reaching a premature stop codon.

In order to find these protein changes , we used *RegEx* in Python to find these regular expressions.

```
pattern = rf"p\.{ch}\d+.*"
for prot in protein_changes:
    if isinstance(prot, str):
        prot_name = re.findall(pattern, prot)
```

- **p.?** : This means that the impact of the mutation is unknown.
- **MLL_PTD** : This refers to the amino acid Lysine.
- **FLT3_ITD** : This refers to the amino acid Tyrosine.
- * : This refers to a non sens mutation.
- We also defined a function to detect the **missense mutations**.

GENE : The gene affected by **the mutation.**

For the genes, we had begun an in-depth exploration of the Gene Ontology using biology libraries such as *myGene* to extract biological and molecular information from each gene.

However, the presence of 124 different genes made the task difficult and uninformative.

Gene Ontology : A bioinformatic project designed to structure the description of genes and gene products within the framework of an **ontology common to all species**.

```
import mygene
genes = list(mol_df["GENE"].unique())
mg = mygene.MyGeneInfo()
results = mg.querymany(genes, scopes="symbol", fields='
    go', species="human")
df_pathways = pd.DataFrame(results)
```

```
mlb = MultiLabelBinarizer()
go_matrix = mlb.fit_transform(gene_to_go.values())
go_df = pd.DataFrame(go_matrix, index=gene_to_go.keys(),
     columns=mlb.classes_)

# 5. Filter out infrequent GO terms (threshold = 5 genes
    )
min_gene_count = 5
filtered_go_df = go_df.loc[:, (go_df.sum(axis=0) >=
    min_gene_count)]
```

With this idea, we had **100 new sparse columns which made the dataset very large.**

After several tests, we decided to encode the **Gene category with a binaryencoder.**

New features :

- mutations_per_gene
- mutations_per_gene_effect
- mutations_per_region

4.2 Clinical dataset Preprocessing

Cytogenetics : Cytogenetics refers to the study of tissue, blood, blood marrow, or culture cells in a laboratory, using banding or manipulating techniques to look for changes in the chromosomes, **including broken, missing , rearranged, or extra chromosomes.**

In order to read a sequence of mutations in cytogenetics, it is essential to refer to a nomenclature. The nomenclature used here is the **ISCN's nomenclature.**

The International System for Human Cytogenomic Nomenclature (ISCN) is an indispensable reference volume for human cytogeneticists, molecular biologists, technicians, and students for the interpretation and communication of human cytogenomic nomenclature.

Let's define a few important terms :

- **46,XX** : Normal female karyotype.
- **46,XY** : Normal male karyotype.
- **del** : deletion of a chromosome fragment.
- **ins** : inserting material into a chromosome.
- **inv** : a fragment has changed orientation within the chromosome itself.
- **t** : translocation

Example :

Karyotype Interpretation: `46,XY,del(20)(q12)[2]/46,XY[18]`

- **46,XY:** Normal male karyotype — 46 chromosomes including one X and one Y.
- **del(20)(q12): A deletion on chromosome 20**, specifically on the long arm (q) at band 12. This indicates that a portion of chromosome 20 is missing.
- **[2]:** This abnormal clone (with the deletion) was observed in 2 cells.
- **/46,XY[18]:** A second, normal cell line with a standard male karyotype was **observed in 18 cells.**

The main function to search for different anomalies :

```
def cyto_regex(cyto, reg_expr):
"""
Check if a cytogenetic string matches a regular
    expression pattern.

Parameters:
-----------
cyto : str
    Cytogenetic string to check
reg_expr : str
    Regular expression pattern to match

Returns:
--------
int
    1 if pattern matches, 0 otherwise
"""
if cyto is not None:
    if re.search(reg_expr, cyto, re.IGNORECASE):
        return 1
return 0
```

```
# Gender identification from karyotype
cl_df = map_lambda(cl_df, "CYTOGENETICS", "is_a_Man",
    is_male_karyotype, pl.Int32)
cl_df = map_lambda(cl_df, "CYTOGENETICS", "is_a_Female",
     is_female_karyotype, pl.Int32)

# Cytogenetic abnormalities detection

# Deletion anomalies
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_deletion_anomaly", lambda x: cyto_regex(x,
    reg_expr=r'del'), pl.Int32)

# Inversion anomalies
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_inversion_anomaly", lambda x: cyto_regex(x,
    reg_expr=r'inv'), pl.Int32)

# Addition anomalies
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_added_anomaly", lambda x: cyto_regex(x, reg_expr=r
    'add'), pl.Int32)

# Common chromosomal abnormalities

# Down Syndrome (Trisomy 21)
```

```
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_down_syndrome", lambda x: cyto_regex(x, reg_expr=r
    '\+21'), pl.Int32)

# Monosomy 7
cl_df = map_lambda(cl_df, "CYTOGENETICS", "is_monosomy",
     lambda x: cyto_regex(x, reg_expr=r'\-7'), pl.Int32)

# Partial deletion of 7th chromosome
cl_df = map_lambda(cl_df, "CYTOGENETICS", "is_7_deleted"
    , lambda x: cyto_regex(x, reg_expr=r'del\(7\)'), pl.
    Int32)

# Trisomy 8
cl_df = map_lambda(cl_df, "CYTOGENETICS", "is_trisomy_8"
    , lambda x: cyto_regex(x, reg_expr=r'\+8'), pl.Int32)

# Other structural abnormalities

# Isochromosome
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    iso_chromosome", lambda x: cyto_regex(x, reg_expr=r"i
    \(\d+\)|iso\(\d.+\)"), pl.Int64)

# Derived chromosome
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_derived_chromosome", lambda x: cyto_regex(x,
    reg_expr=r"der\(\d+\)"), pl.Int64)

# Sex chromosome abnormalities
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_lost_sex_chromosome", lambda x: cyto_regex(x,
    reg_expr=r"\-[xy]"), pl.Int64)

# Numerical abnormalities
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_added_chromsome", lambda x: cyto_regex(x, reg_expr
    =r",\+\d+"), pl.Int64)
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_deleted_chromsome", lambda x: cyto_regex(x,
    reg_expr=r",\-\d+"), pl.Int64)

# Insertion abnormalities
cl_df = map_lambda(cl_df, "CYTOGENETICS", "
    is_inserted_chromsome", lambda x: cyto_regex(x,
    reg_expr=r"ins"), pl.Int64)

# Translocation anomaly
cl_df = map_lambda(cl_df , "CYTOGENETICS" , "
    is_translocated_anomaly" , lambda x: cyto_regex(x ,
```

```
        reg_expr=r't' ) , pl.Int32)

    # Complex karyotype detection
    cl_df = map_lambda(cl_df, "CYTOGENETICS", "
        is_complex_karyo", is_complex, pl.Int64)
    # Handle missing cytogenetic data
    cl_df = is_missing_cytogenetics(cl_df)
```

As for null values and outliers, the strategies used datasets were the same.

Null values : For null values, we decided to implement a robust process rather than replacing with the median or mean . **To do this, we used IterativeImputer from Sickit-Learn.**

```
    # Apply imputation
    imputer = IterativeImputer(estimator=estimator,
        random_state=42)
    imputed_data = imputer.fit_transform(sub_pd)

    # Convert back to polars
    imputed_pl = pl.DataFrame(imputed_data, schema=sub_df.
        columns)

    # Replace nulls in original dataframe
    df = df.with_columns([imputed_pl[col].alias(col) for col
         in column_list])

    return df
```

Outliers : We used the IQR-Method to detect potential outliers.

```
    q1 = df.select(pl.col(cl).quantile(0.25)).item()
    q3 = df.select(pl.col(cl).quantile(0.75)).item()

    iqr = q3 - q1


    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr


    # Winsorisation : capped values
    df = df.with_columns(
            pl.when(pl.col(cl) < lower_bound).then(
                lower_bound)
            .when(pl.col(cl) > upper_bound).then(upper_bound
                )
            .otherwise(pl.col(cl))
```

```
            .alias(cl))
```

Finally, we used binary encoding for the different categories, **as it doesn't create order or explode the number of columns created**, and we used min-max normalization for the different numerical columns.

5 Feature Selection: Identifying Key Prognostic Factors

After this pre-processing work, we now have **60 columns in our dataset.** Now it's time to study the relationships between the different variables, in order to select **the features that make the most sense.**

5.1 Variance Threshold

The Variance Threshold technique in machine learning is a feature selection method that **removes low-variance features from your dataset.** It's a simple and fast way to eliminate features that do not vary much across samples and are **therefore unlikely to be useful for distinguishing between target classes.**

We used the $VarianceThreshold$ from sklearn.feature_selection.

Conclusion : We computed the variances of the different columns and found that the variance was very low. **On average, we had a variance of 0.1. So it was difficult to apply this method.**

5.2 Pearson Correlation

The Pearson correlation coefficient, often just called Pearson's r, is a measure of the linear relationship between two variables.

To get an idea of the linear relationships that may exist between variables, we used the Pearson's r.

Conclusion : We noticed a large positive correlation between many columns. For example: **the coefficient of relationship between "is_indel" and "is_frameshit" is 0.93**.

Global Conclusion : In this context of survival analysis, it is important to understand that the survival analysis is very specific .**To this end, we're going to use a survival model for feature selection.**

6 Survival Analysis

Survival analysis is a branch of statistics and machine learning that deals with the analysis of time-to-event data. The "event" can be any occurrence of interest such as death, system failure, customer churn, or recovery from illness. **The main goal is to estimate the time until the event occurs and understand factors that influence it.**

6.1 Key Concepts

- **Survival Time (T)**: The time duration until the event occurs.
- **Censoring**: A data point is censored if the event has not occurred by the end of the study period. **For example, if a patient is still alive at the end of the study, we know only that their survival time exceeds the observation period.**
- **Survival Function ($S(t)$)**: Represents the probability that the time to event T exceeds a certain time t:

$$S(t) = P(T > t)$$

- **Hazard Function ($\lambda(t)$)**: Describes the instantaneous rate at which the event occurs at time t, **given survival up to that time:**

$$\lambda(t) = \lim_{\Delta t \to 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}$$

6.2 Common Models

- **Kaplan-Meier Estimator**: A non-parametric method to estimate the survival function from censored data. It produces a step-wise survival curve.
- **Log-Rank Test**: A statistical test to compare survival distributions of two or more groups.
- **Cox Proportional Hazards Model**: A semi-parametric model that assesses the effect of covariates (e.g., age, treatment) on the hazard function. **It assumes the hazard ratio between groups is constant over time.**
- **Parametric Models**: Assumes a specific distribution for survival times (e.g., Exponential, Weibull, Log-normal).

6.3 Examples

1. **Medical Research**: Estimating the survival time of patients after receiving a specific treatment, accounting for censored data when patients drop out or the study ends.
2. **Engineering (Reliability Analysis)**: Estimating the time until a machine component fails. Components still working at the end of testing are considered censored.
3. **Customer Churn Prediction**: Predicting how long a customer will remain subscribed to a service. Customers who are still active at the time of analysis are censored.

6.4 Overall Survival of Myeloid Leukemia Patients

In our context of analyzing Overall Survival of Myeloid Leukemia Patients , we have two key results provided:

1. **OS_YEARS (Survival Time)**: Overall survival time in years since diagnosis.
2. **OS_STATUS (Censoring)**: Survival status indicator, where 1 indicates death and 0 indicates that the patient was alive at the last follow-up.

To perform this survival analysis correctly, we'll use the Sickit-Survival library.

7 Cox Proportional Hazards Model

7.1 Mathematical understanding

$$\lambda(t \mid X) = \lambda_0(t) \cdot \exp\left(\beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p\right)$$

where:

- $\lambda(t \mid X)$ is the **hazard function** at time t given the covariates $X = (X_1, X_2, \ldots, X_p)$,
- $\lambda_0(t)$ is the **baseline hazard function** at time t, representing the hazard when all covariates are zero,
- $\beta_1, \beta_2, \ldots, \beta_p$ are the **regression coefficients** corresponding to each covariate,
- $X_1, X_2, \ldots, X_p$ are the **covariates** (predictor variables) influencing the hazard.

The model assumes that the effect of the covariates is multiplicative with respect to the baseline hazard, and the hazard ratios between individuals remain **proportional over time**.

7.2 CoxPHFitter

As mentioned above, we will use the CoxPH model to perform the feature selection step.

CoxPHFitter : A class in the lifelines Python library used to fit **a Cox Proportional Hazards (CoxPH) model for survival analysis.**

```
cph = CoxPHFitter(penalizer=0.01)
cph.fit(df_filtered_pd , duration_col='time', event_col=
    'event')

# Summary
cph.print_summary()
summary_df = cph.summary
```

This summary provides us with several pieces of information, but the one we're most interested in is the **p-value.**

7.3 Computation of p-values in Cox Proportional Hazards Model

In the Cox Proportional Hazards model, each covariate X_j has an estimated coefficient $\hat{\beta}_j$. The p-value tests the null hypothesis:

$$H_0 : \beta_j = 0,$$

which means the covariate has no effect on the hazard.

The steps to compute the p-value are as follows:

1. Estimate the coefficient $\hat{\beta}_j$ by maximizing the partial likelihood.
2. Calculate the standard error $SE(\hat{\beta}_j)$ from the inverse of the observed information matrix (the Hessian of the log partial likelihood).
3. Compute the Wald statistic:

$$Z_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}.$$

4. Under the null hypothesis and asymptotic normality, Z_j approximately follows a standard normal distribution.
5. The two-sided p-value is computed as:

$$p = 2 \times P(|Z| > |Z_j|),$$

where P is the cumulative distribution function (CDF) of the standard normal distribution.

In summary:

- The p-value tests whether the coefficient is significantly different from zero.
- It is derived from the **Wald test**, which uses the coefficient estimate and its standard error.

- **Small p-values (e.g., < 0.05) indicate that the covariate significantly affects the hazard.**

Note: Besides the Wald test, other tests like the likelihood ratio test and the score test can also be used to assess significance in Cox models. By default, `CoxPHFitter` reports the Wald test p-values.

These tests rely on large-sample approximations and thus are more reliable with sufficient data.

```
insignificant_vars = summary_df[summary_df['p'] > 0.01].
    index.tolist()

print("Non-significant variables (p > 0.01) :",
    insignificant_vars)

print(f"Size {len(insignificant_vars)}")
```

Conclusion : We have successfully sorted **40 variables using the p-value strategy.** We now have 19 columns.

7.4 Concordance Index in Survival Analysis

The **concordance index** (C-index) is a popular metric to evaluate the predictive accuracy of survival models. It measures the agreement between the predicted risk scores and the observed survival outcomes, accounting for censoring.

1. Concordance Index with Censoring: `concordance_index_censored`

This is a commonly used version of the C-index that properly handles right-censored survival data. It evaluates all comparable pairs of subjects where one can determine which subject experienced the event first. The index is computed as the proportion of all usable pairs in which the predictions and outcomes are concordant.

Formally, for pairs (i, j) where i has an event time T_i and j has T_j, and one time is strictly less than the other and uncensored, the pair is *concordant* if the predicted risk scores r_i and r_j correctly rank the subjects (e.g., higher risk score corresponds to shorter survival).

$$C - index = \frac{Numberofconcordantpairs + 0.5 \times Numberoftiedpairs}{Numberofcomparablepairs}$$

2. IPCW Concordance Index (Inverse Probability of Censoring Weighting)

The IPCW method adjusts the C-index to account for the bias introduced by censoring. Instead of ignoring censored pairs that are incomparable, it weights

each pair inversely proportional to the probability of censoring. This provides a consistent estimate of predictive accuracy even with heavy censoring.

The weights are typically estimated from the censoring distribution (e.g., via Kaplan-Meier), and the IPCW C-index can be expressed as:

$$IPCWC - index = \frac{\sum_{(i,j)} w_{ij} \cdot I(concordant_{ij})}{\sum_{(i,j)} w_{ij}}$$

where w_{ij} are the inverse probability weights.

Summary:

- `concordance_index_censored` is a nonparametric, standard C-index that handles censoring by ignoring incomparable pairs.
- The **IPCW C-index incorporates censoring weights to reduce bias,** especially useful with substantial censoring.
- Both are widely used to assess the discrimination ability of survival models like CoxPH.

7.5 Integrated Brier Score (IBS)

1. Brier Score is a strictly proper scoring rule that measures the accuracy of probabilistic predictions. For unidimensional predictions, it is strictly equivalent to the mean squared error as applied to predicted probabilities.

The Brier score can be thought of as a **cost function.**

More precisely, across all items $i \in \{1 \dots N\}$ in a set of N predictions, the Brier score measures the mean squared difference between:

- The predicted probability assigned to the possible outcomes for item i
- The actual outcome o_i

The most common formulation of the Brier score is:

$$BS = \frac{1}{N} \sum_{t=1}^{N} (f_t - o_t)^2$$

where:

- N is the number of predictions,
- f_t is the predicted probability at time t,
- o_t is the actual outcome at time t (1 if the event occurred, 0 otherwise).

2. The Integrated Brier Score (IBS) is a performance measure for survival models. It summarizes the time-dependent Brier score over a time interval $[t_1, t_{max}]$, taking into account both prediction error and censoring.

The integrated Brier score is defined as:

$$IBS = \int_{t_1}^{t_{max}} BS^c(t)\, dw(t)$$

where $BS^c(t)$ is the time-dependent Brier score at time t, and $w(t) = \frac{t}{t_{max}}$ is a weighting function that increases linearly with time. The integral is typically estimated using the trapezoidal rule.

The time-dependent **Brier score** $BS^c(t)$ at a specific time t is defined as:

$$BS^c(t) = \frac{1}{n}\sum_{i=1}^{n}\left[\frac{I(y_i \leq t \wedge \delta_i = 1)}{\hat{G}(y_i)}(0 - \hat{S}(t \mid \mathbf{x}_i))^2 + \frac{I(y_i > t)}{\hat{G}(t)}(1 - \hat{S}(t \mid \mathbf{x}_i))^2\right]$$

where: - $\hat{S}(t \mid \mathbf{x}_i)$ is the predicted survival probability for instance i at time t, - $\hat{G}(t)$ is the Kaplan–Meier estimate of the censoring distribution, - δ_i is the event indicator (1 if event occurred, 0 if censored), - $I(\cdot)$ is the indicator function.

The IBS provides an overall measure of how well a model predicts survival probabilities across a range of time points. **Lower IBS values indicate better predictive accuracy.**

IBS Values

- 0 : Model predictions match **reality exactly**
- ≤ 0.25 : Generally **considered good performance** (depending on dataset and context)
- $0.25 - 0.5$: **Moderate performance**, possibly room for improvement
- ≥ 0.5 : Poor model — **worse than trivial guessing**

```
# Integrated Brier Score
survs_train = model.predict_survival_function(X_train)
survs_test = model.predict_survival_function(X_test)

times = np.arange(0,7) # 0 to 7 years
preds_train = np.asarray([[fn(t) for t in times] for fn
    in survs_train])
preds_test = np.asarray([[fn(t) for t in times] for fn
    in survs_test])

ib_score_train = integrated_brier_score(y_train_split,
    y_train_split,preds_train,times)
ib_score_test = integrated_brier_score(y_train_split,
    y_test_split,preds_test,times)
```

8 Evaluation of different survival models

Remark : It's essential that the target dataframe is in a particular form that is conducive to survival analysis.

```
from sksurv.util import Surv

y_train = y_train.rename(
{"OS_YEARS" : "time" ,
 "OS_STATUS" : "event"}
)
y_train_pd = y_train.to_pandas()

y_train_pd = Surv.from_dataframe('event' , 'time' ,
    y_train_pd)
```

Output :array([(True, 1.11506849), (False, 4.92876712), (False, 2.04383562), ..., (False, 1.99726027), (True, 0.09589041), (False, 2.29041096)], shape=(3323,), dtype=[('event', '?'), ('time', '<f8')])

8.1 Models

```
X_train , X_test , y_train , y_test = train_test_split(X
     , y , test_size=0.2 , random_state=42)

models = {"CoxPH": CoxPHSurvivalAnalysis(),
    "Random Survival Forest": RandomSurvivalForest(
        random_state=42),
    "GradientBoostingSurvivalAnalysis" :
        GradientBoostingSurvivalAnalysis()
}
```

8.2 Concordance Index (IPCW) Evaluation Results

- **Cox Proportional Hazards Model:**
  - IPCW Concordance Index on training set: **0.67**
  - IPCW Concordance Index on test set: **0.69**
- **Random Survival Forest:**
  - IPCW Concordance Index on training set: **0.80**
  - IPCW Concordance Index on test set: **0.77**
- **Gradient Boosting Survival Model:**
  - IPCW Concordance Index on training set: **0.69**
  - IPCW Concordance Index on test set: **0.70**

8.3 HyperParameters tuning with GridSearchCV

```
model_params = [
{
    "name": "CoxPH",
    "estimator": CoxPHSurvivalAnalysis(),
    "params": {
        "alpha": [0.001, 0.01, 0.02, 0.05],
        "ties": ["breslow", "efron"],
        "n_iter": [100, 200, 500, 1000]
    }
},
{
    "name": "RSF",
    "estimator": RandomSurvivalForest(n_jobs=-1),
    "params": {
        "n_estimators": [50, 100, 150, 200, 300],
        "min_samples_split": [6, 8, 10, 15, 20],
        "min_samples_leaf": [3, 5, 7, 12, 15, 20],
        "max_features": ["sqrt", "log2"]
    }
}
{
    "name": "GradientBoost",
    "estimator": GradientBoostingSurvivalAnalysis(
        random_state=42),
    "params" :{
        "learning_rate": [0.1, 0.5,1,1.5],
        "n_estimators": [100, 150,200],
        "max_depth": [1, 3, 5]
    }
}
```

```
]
    kf = KFold(n_splits=10, shuffle=True, random_state=42)

    def ipcw_score(estimator, X, y):
        prediction = estimator.predict(X)
        result = concordance_index_ipcw(y, y, prediction,
            tau=7)
        return result[0]
```

Training time : 606m 5.2s

Model	Best Parameters	CV Score (IPCW)
Cox Proportional Hazards (CoxPH)	`alpha = 0.001, n_iter = 100, ties = 'breslow'`	0.669
Random Survival Forest (RSF)	`max_features = 'sqrt', min_samples_leaf = 3, min_samples_split = 6, n_estimators = 200`	0.813
Gradient Boosting Survival Analysis	`learning_rate = 1, max_depth = 5, n_estimators = 200`	0.873

Table 1: Best parameters and cross-validation (IPCW) scores for survival models

1. Concordance Index (IPCW) :

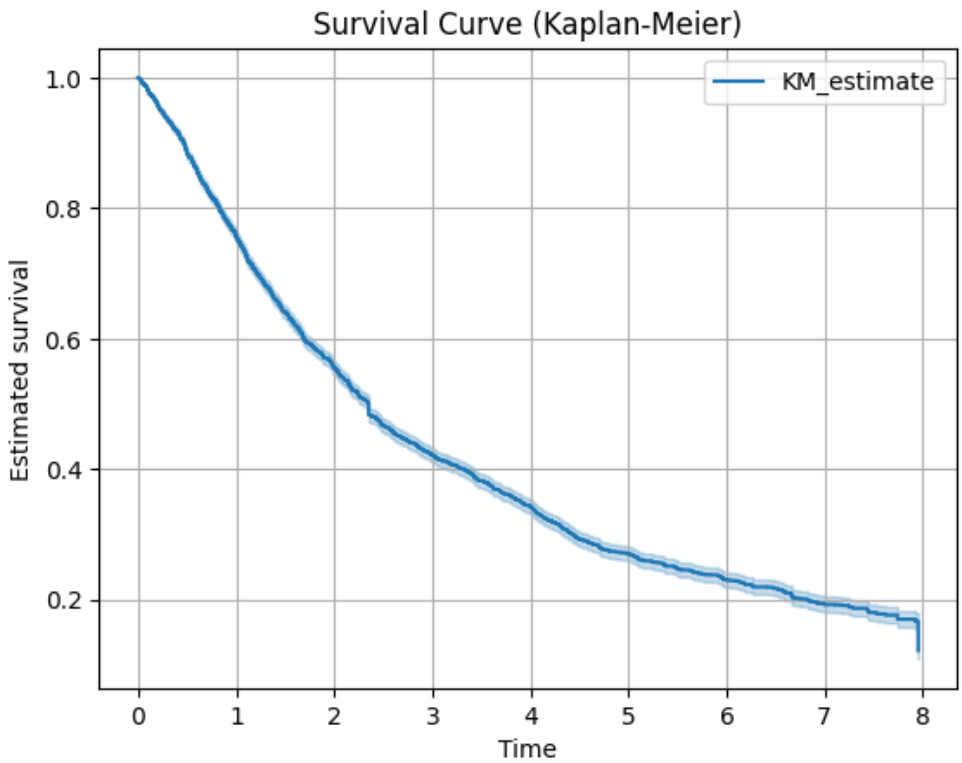
- **Cox Proportional Hazards Model:**
  - IPCW Concordance Index on training set: **0.67**
  - IPCW Concordance Index on test set: **0.69**
- **Random Survival Forest:**
  - IPCW Concordance Index on training set: **0.91**
  - IPCW Concordance Index on test set: **0.83**
- **GradientBoostingSurvivalAnalysis**
  - IPCW Concordance Index on training set: **0.95**
  - IPCW Concordance Index on test set: **0.87**

2. Integrated Brier Score :

- **Cox Proportional Hazards Model:**
  - Integrated Brier Score on training set: **0.1666**
  - Integrated Brier Score on test set: **0.1603**
- **Random Survival Forest:**
  - Integrated Brier Score on training set: **0.0426**
  - Integrated Brier Score on test set: **0.0935**
- **GradientBoostingSurvivalAnalysis**
  - Integrated Brier Score on training set: **0.0379**
  - Integrated Brier Score on test set: **0.0770**

9 Clinical Interpretation and Survival Curve Analysis

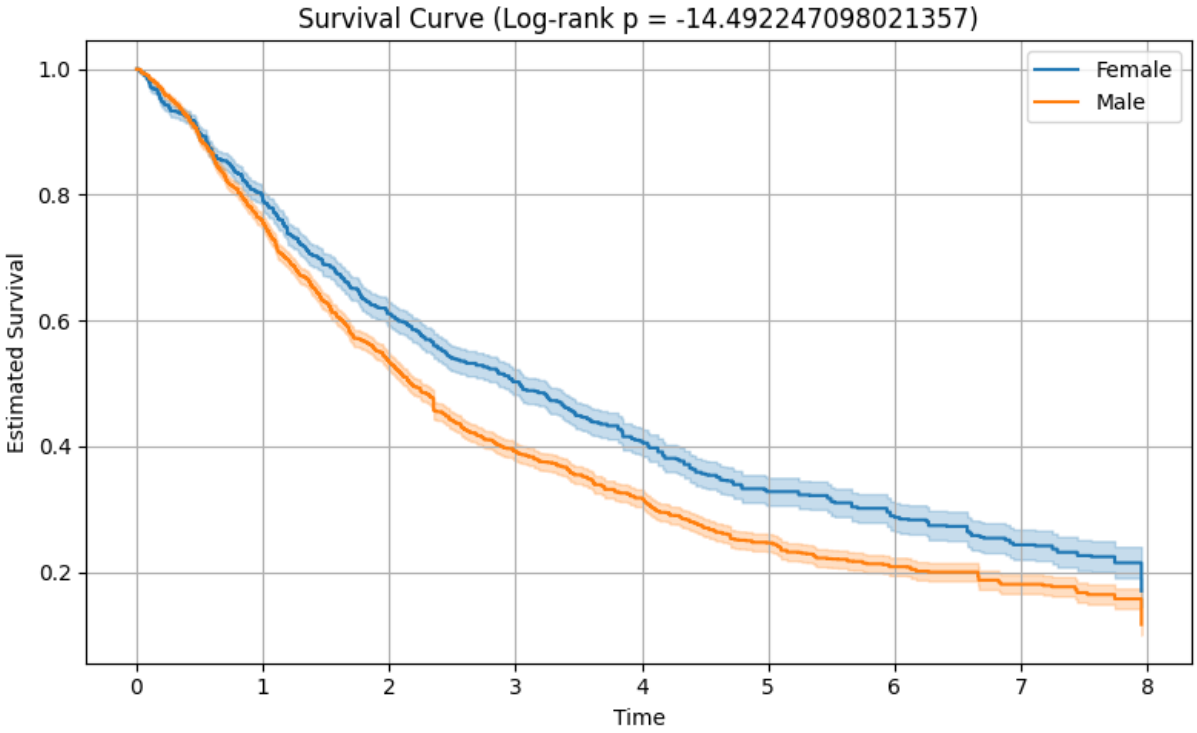
9.1 Survival Curve



Commentary : 50% of individuals are estimated to have **survived past year 2.5.**

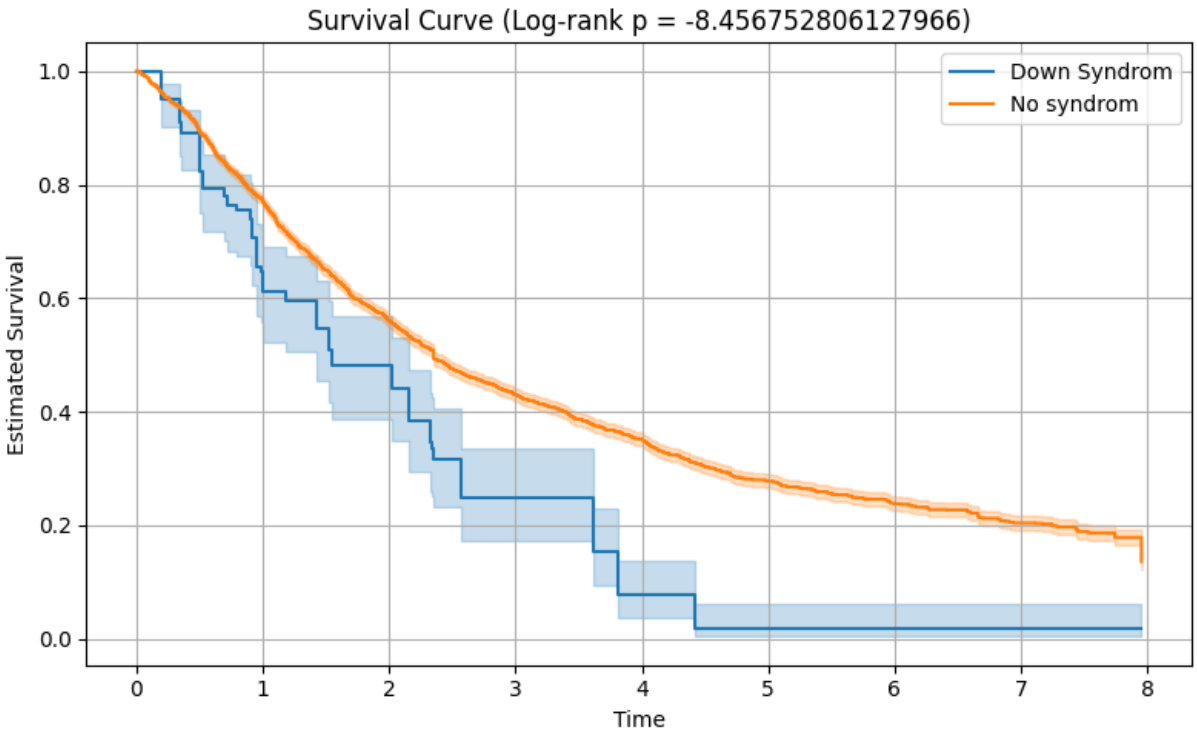
We will now use the **Log-rank-test** to analyze survival between two different groups and see the impact of the presence of certain anomalies.

9.2 Male Female Survival Curve



Commentary : Women survive illness longer than men.

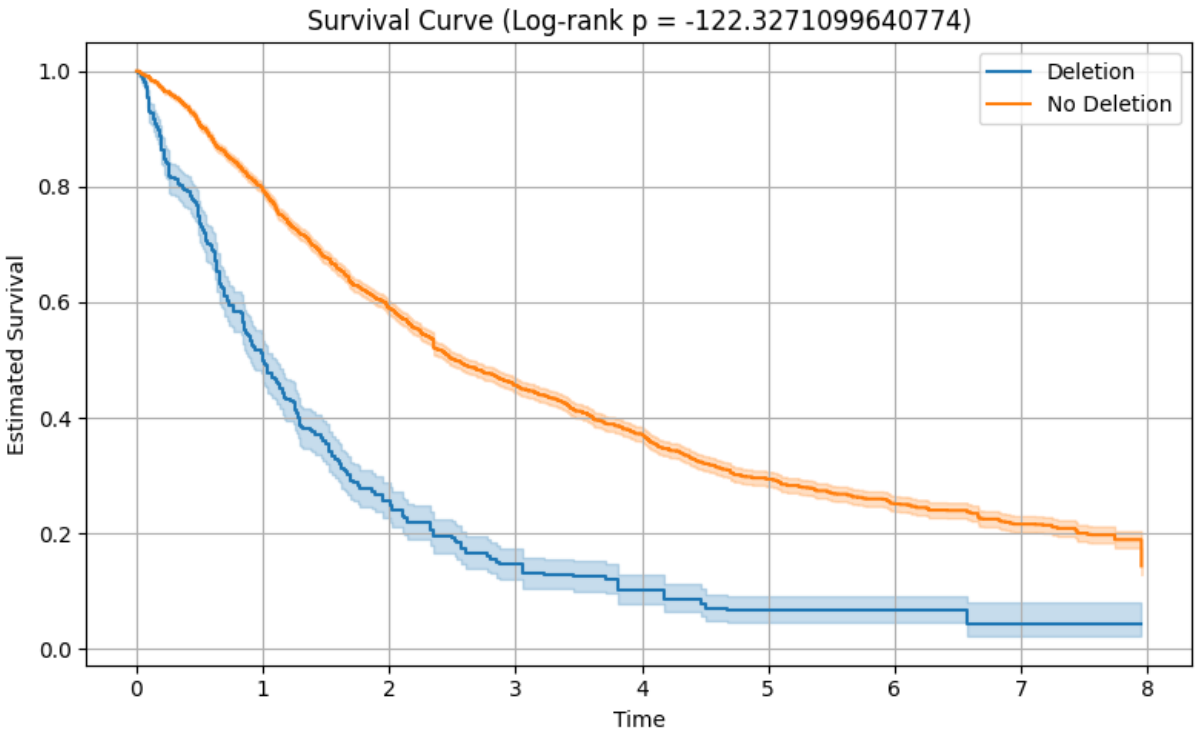
9.3 Down Syndrome Survival



Commentary : Individuals with Down's syndrome survive for shorter periods than those without.

For instance , after 4 years only 15% inviduals with Down's syndrome survived compared to 37%.

9.4 Deletion anomaly Survival



Commentary : The deletion anomaly has a major impact on patient survival.

- - END - -